

EtherCAT과 CAN FD 완벽 가이드

목차

- 1. EtherCAT 개요
- 2. CAN FD 개요
- 3. 유사 통신 방식과의 비교
- 4. 발전 역사와 현재 사용 이유
- 5. EtherCAT과 CAN FD 연결 방법

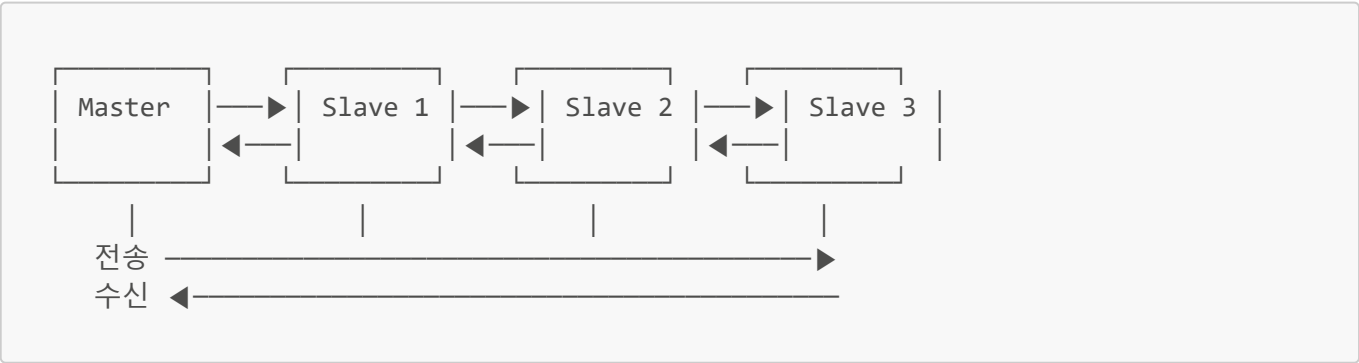
1. EtherCAT 개요

1.1 EtherCAT이란?

****EtherCAT (Ethernet for Control Automation Technology)****은 Beckhoff Automation에서 개발한 산업용 이더넷 프로토콜입니다.

항목	내용
개발사	Beckhoff Automation (독일)
개발년도	2003년
표준화	IEC 61158, IEC 61784
물리 계층	100BASE-TX (표준 이더넷)
토폴로지	라인, 트리, 스타, 링
최대 속도	100 Mbps
사이클 타임	< 100 μ s (매우 빠름)

1.2 EtherCAT 동작 원리

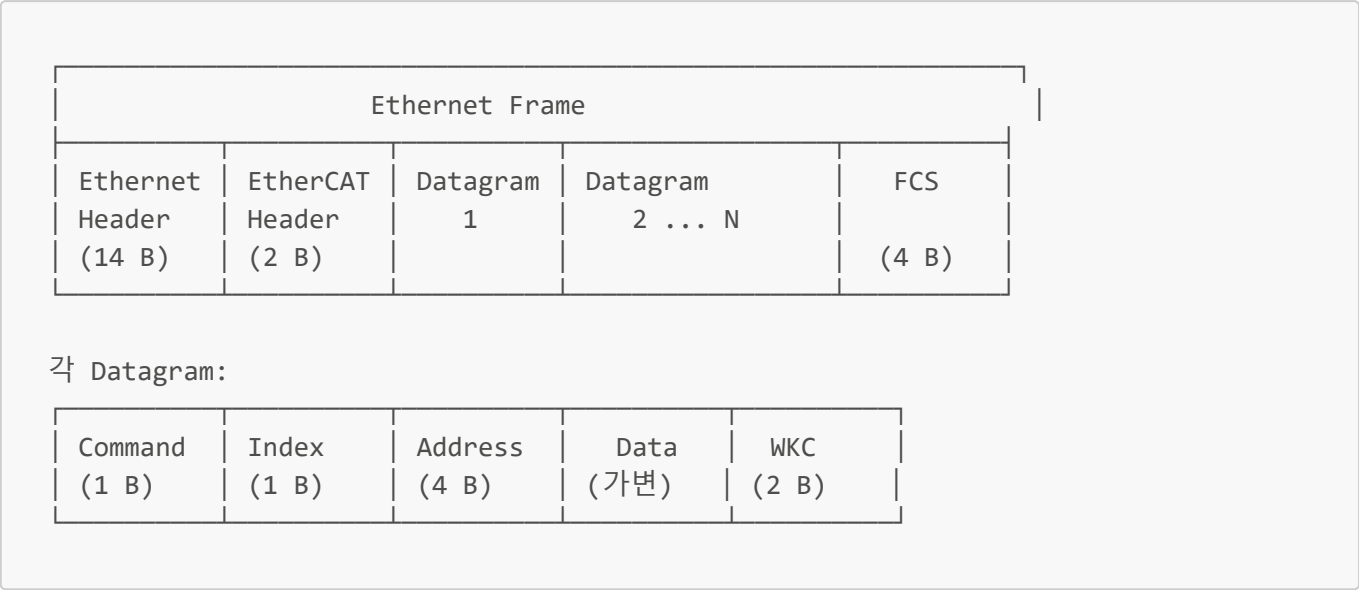


핵심 특징: "On-the-fly" 처리

- 이더넷 프레임이 각 슬레이브를 통과하면서 데이터를 읽고/쓰기
- 프레임이 마지막 슬레이브에 도달하면 자동으로 되돌아옴
- 각 슬레이브는 프레임이 통과하는 동안 필요한 데이터만 추출/삽입

- 지연 시간: 슬레이브당 약 1μs 이하

1.3 EtherCAT 프레임 구조



1.4 EtherCAT 장점

장점	설명
초고속	사이클 타임 < 100μs, 1000개 I/O 처리에 30μs
효율성	하나의 프레임으로 모든 슬레이브 통신
유연한 토폴로지	라인, 트리, 스타, 링 구조 지원
표준 이더넷	표준 이더넷 케이블/커넥터 사용
핫 커넥트	운영 중 슬레이브 추가/제거 가능
진단 기능	상세한 에러 진단 및 위치 파악

1.5 EtherCAT 적용 분야

- 로봇 제어 시스템
- CNC 공작기계
- 반도체 제조 장비
- 패키징 기계
- 인쇄 기계
- 모션 컨트롤 시스템

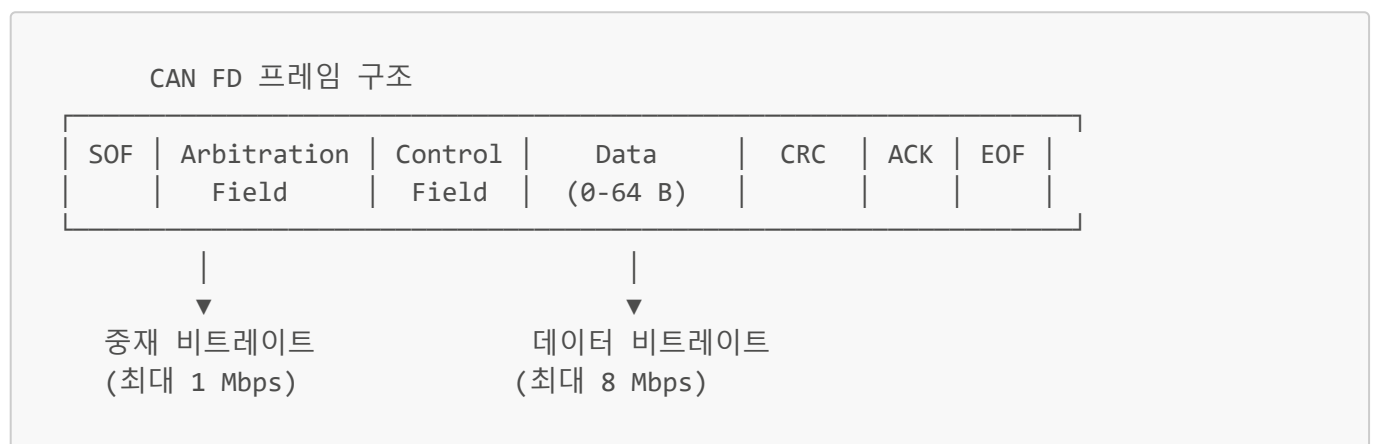
2. CAN FD 개요

2.1 CAN FD란?

CAN FD (Controller Area Network with Flexible Data-rate)는 기존 CAN 2.0의 한계를 극복하기 위해 개발된 차세대 CAN 프로토콜입니다.

항목	CAN 2.0	CAN FD
개발사	Bosch (1986)	Bosch (2012)
표준화	ISO 11898-1	ISO 11898-1:2015
최대 데이터	8 바이트	64 바이트
비트레이트	최대 1 Mbps	중재: 1 Mbps, 데이터: 최대 8 Mbps
호환성	-	CAN 2.0과 하위 호환

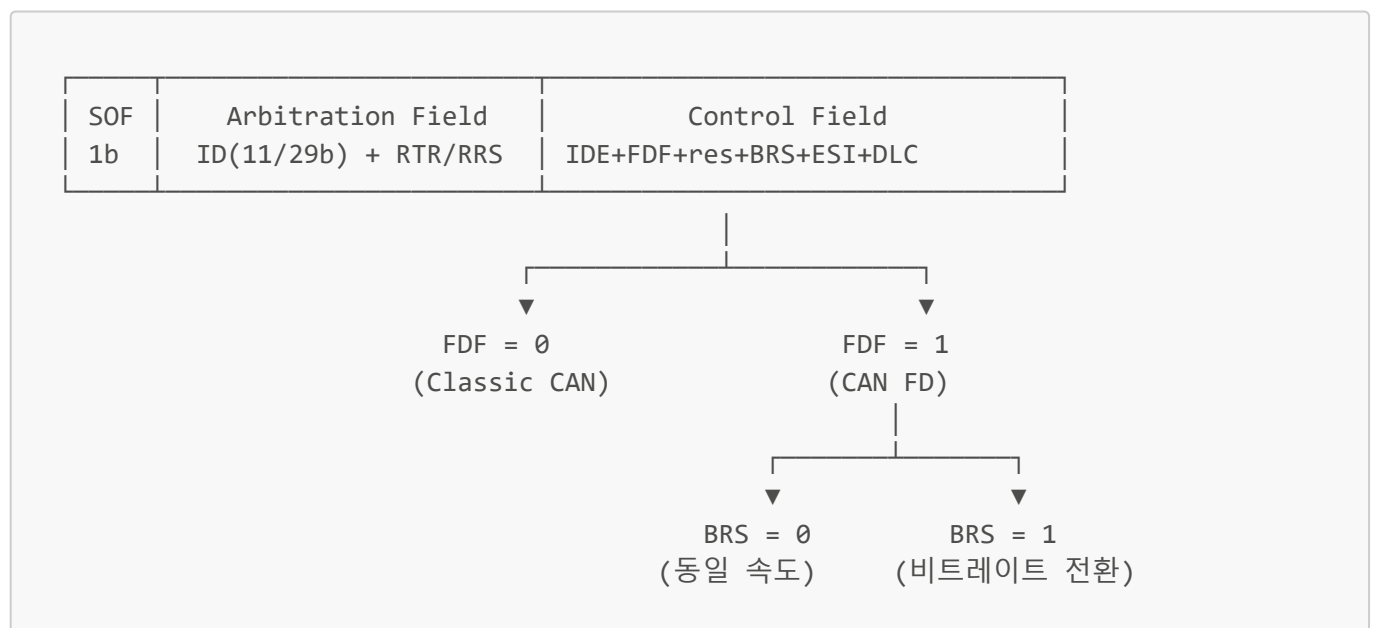
2.2 CAN FD 동작 원리



Flexible Data-rate의 의미:

- 중재(Arbitration) 구간: 느린 속도 (호환성 유지)
- 데이터(Data) 구간: 빠른 속도 (성능 향상)

2.3 CAN FD 프레임 상세



주요 필드 설명:

- **FDF (FD Format):** CAN FD 프레임 식별
- **BRS (Bit Rate Switch):** 비트레이트 전환 여부

- **ESI (Error State Indicator):** 에러 상태 표시
- **DLC:** 데이터 길이 (0-64 바이트)

2.4 CAN FD DLC 매핑

DLC 값	CAN 2.0 데이터	CAN FD 데이터
0-8	0-8 바이트	0-8 바이트
9	-	12 바이트
10	-	16 바이트
11	-	20 바이트
12	-	24 바이트
13	-	32 바이트
14	-	48 바이트
15	-	64 바이트

2.5 CAN FD 장점

장점	설명
대용량 데이터	최대 64바이트 (CAN 2.0의 8배)
고속 전송	데이터 구간 최대 8 Mbps
하위 호환	CAN 2.0 노드와 공존 가능
효율성	더 적은 프레임으로 더 많은 데이터 전송
CRC 강화	17/21비트 CRC로 에러 검출 강화

2.6 CAN FD 적용 분야

- 자동차 (ADAS, 인포테인먼트)
- 산업 자동화
- 의료 장비
- 농업 기계
- 철도 시스템
- 항공우주

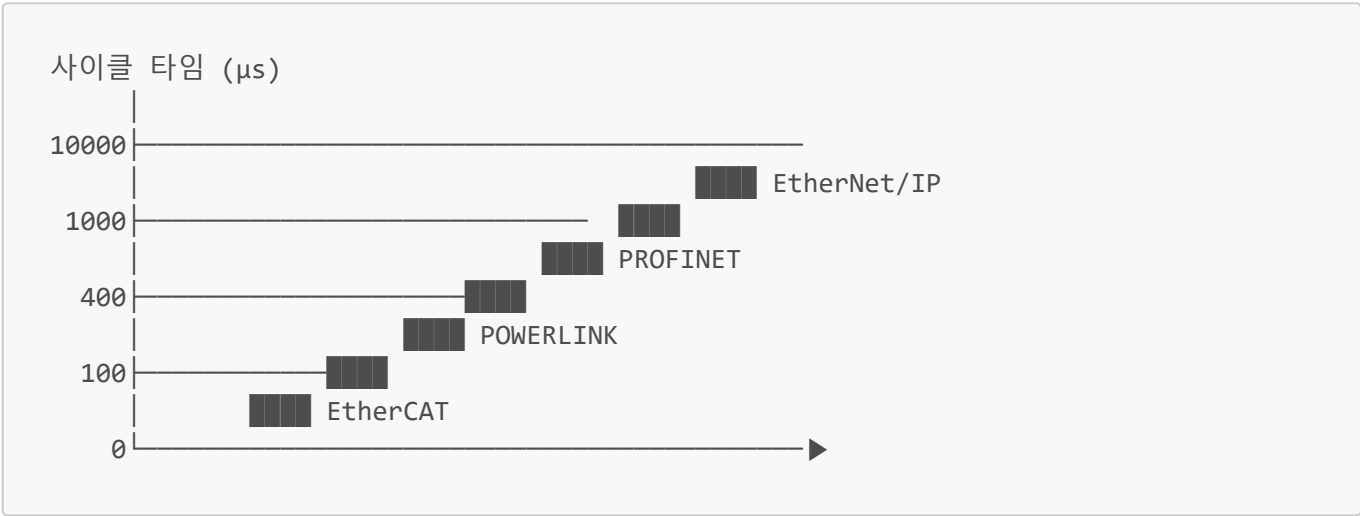
3. 유사 통신 방식과의 비교

3.1 산업용 이더넷 비교

프로토콜	개발사	사이클 타임	토폴로지	특징
EtherCAT	Beckhoff	< 100 μ s	라인/트리/스타/링	On-the-fly 처리, 초고속

프로토콜	개발사	사이클 타임	토폴로지	특징
PROFINET IRT	Siemens	< 1 ms	스타/라인/링	시간 동기화, 폭넓은 지원
EtherNet/IP	ODVA	1-10 ms	스타	표준 TCP/IP 기반, 구현 쉬움
POWERLINK	B&R	< 400 μ s	라인/트리	오픈소스, 시분할
SERCOS III	Bosch Rexroth	< 1 ms	링	모션 컨트롤 특화
CC-Link IE	Mitsubishi	< 1 ms	스타/라인/링	일본 시장 강세

3.2 성능 비교 그래프



3.3 CAN 계열 비교

프로토콜	최대 속도	데이터 크기	특징
CAN 2.0A	1 Mbps	8 바이트	11비트 ID
CAN 2.0B	1 Mbps	8 바이트	29비트 확장 ID
CAN FD	8 Mbps	64 바이트	가변 비트레이트
CAN XL	20 Mbps	2048 바이트	차세대 (개발 중)

3.4 EtherCAT vs CAN FD 비교

항목	EtherCAT	CAN FD
물리 계층	이더넷 (100BASE-TX)	차동 신호 (CAN PHY)
속도	100 Mbps	최대 8 Mbps
토폴로지	라인/트리/스타/링	버스
케이블 길이	최대 100m (노드 간)	최대 40m (8 Mbps)
노드 수	65,535개	약 64개 (실용적)
실시간성	매우 높음 (< 100 μ s)	높음 (메시지 우선순위)

항목	EtherCAT	CAN FD
비용	중-고	저-중
복잡성	높음	중간
주요 용도	고속 모션 컨트롤	차량/분산 제어

4. 발전 역사와 현재 사용 이유

4.1 EtherCAT 역사



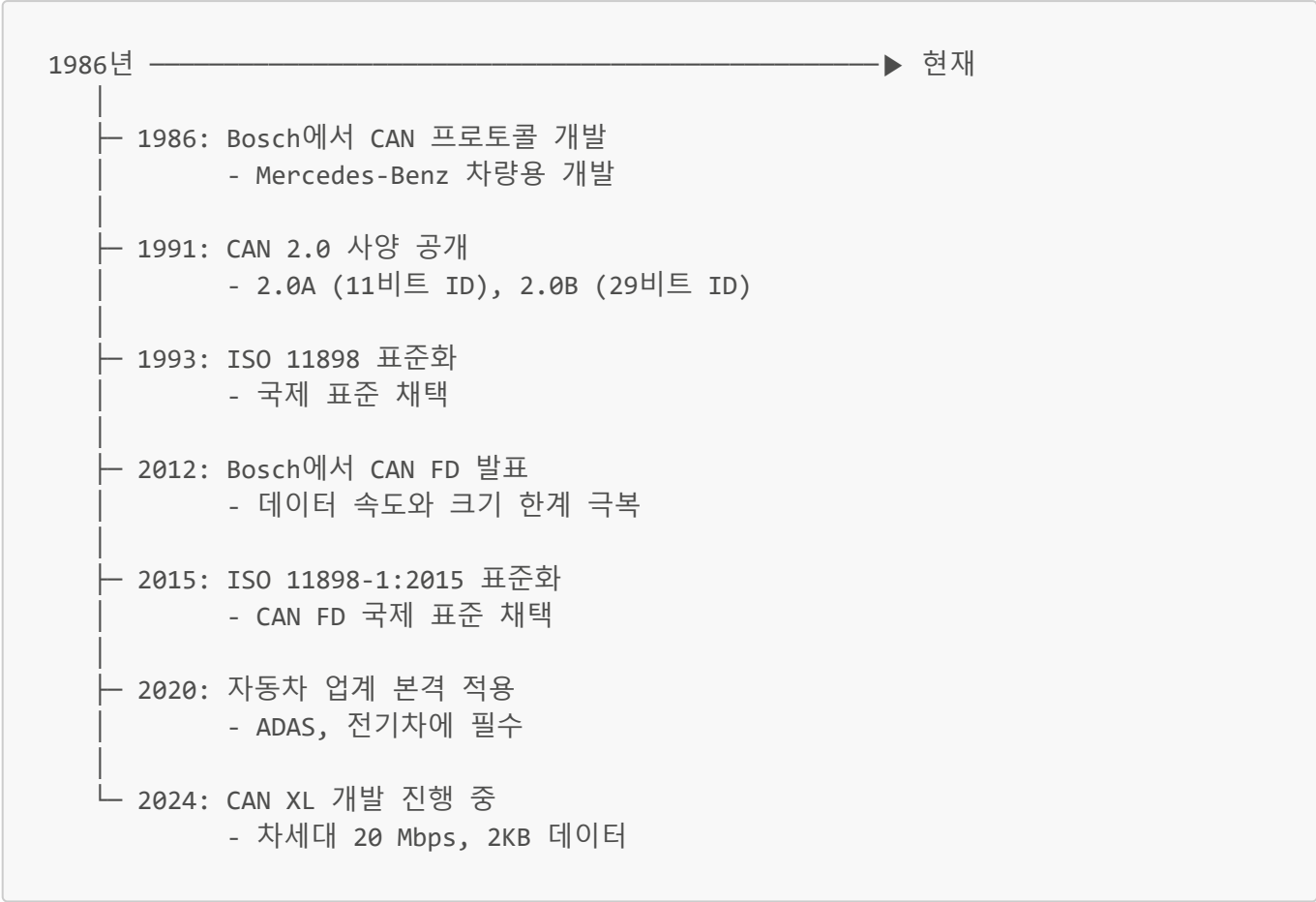
EtherCAT이 처음 개발된 이유:

1. 기존 필드버스(PROFIBUS, DeviceNet)의 속도 한계
2. 증가하는 모션 컨트롤 요구사항
3. 표준 이더넷 기술 활용 필요성
4. 더 많은 축(Axis)의 동기화 필요

현재 사용되는 이유:

1. 성능: 여전히 가장 빠른 산업용 이더넷
2. 생태계: 7,000개 이상 회원사, 풍부한 제품군
3. 개방성: 오픈 표준, 라이선스 비용 없음
4. 호환성: 다양한 토폴로지와 프로토콜 지원
5. 발전성: EtherCAT G로 기가비트 확장

4.2 CAN FD 역사



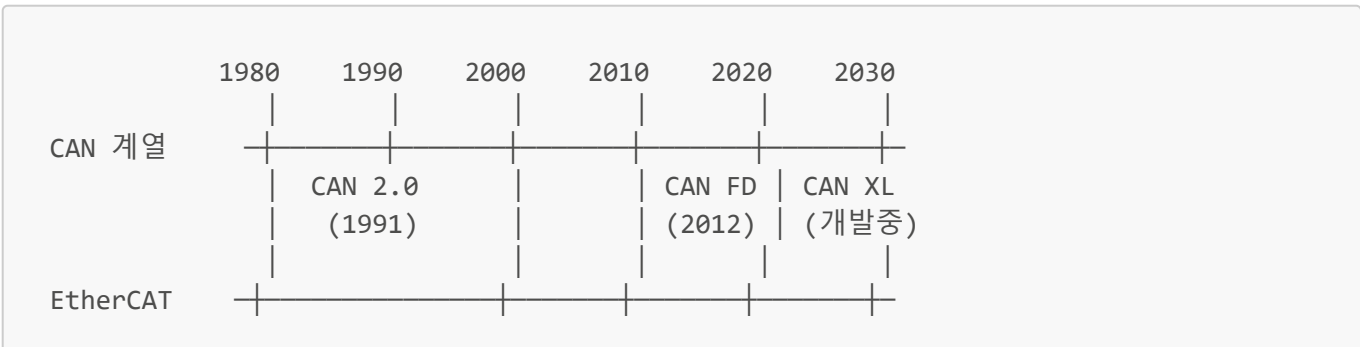
CAN FD가 개발된 이유:

- 1. 자동차 전자장치 급증 (ECU 100개 이상)
- 2. ADAS, 인포테인먼트 대용량 데이터 요구
- 3. 기존 CAN 2.0의 8바이트 한계
- 4. 더 빠른 소프트웨어 업데이트 필요

현재 사용되는 이유:

- 1. 호환성: 기존 CAN 인프라 활용 가능
- 2. 비용 효율: 이더넷 대비 저렴한 구현
- 3. 신뢰성: 검증된 CAN 기술 기반
- 4. 유연성: 기존 CAN과 공존 가능
- 5. 산업 채택: 자동차 OEM 전면 채택

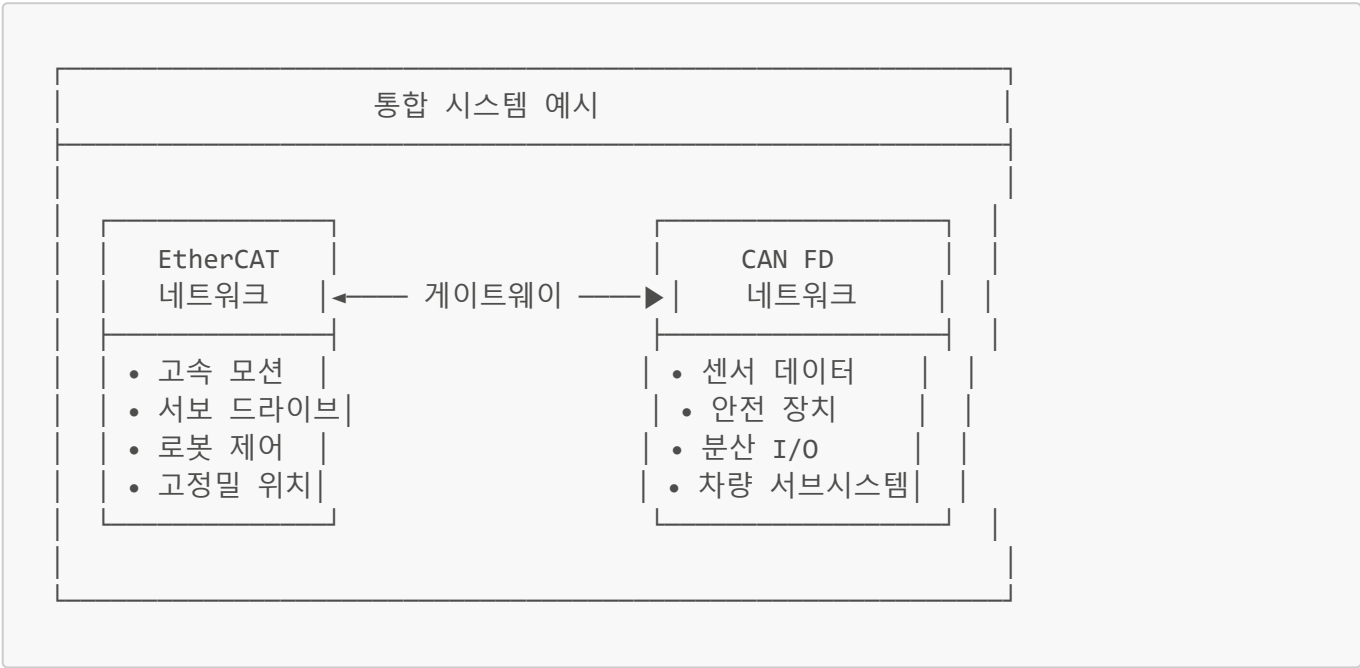
4.3 기술 발전 타임라인 비교



		EtherCAT (2003)	EtherCAT G (2018)	

5. EtherCAT과 CAN FD 연결 방법

5.1 연결이 필요한 이유

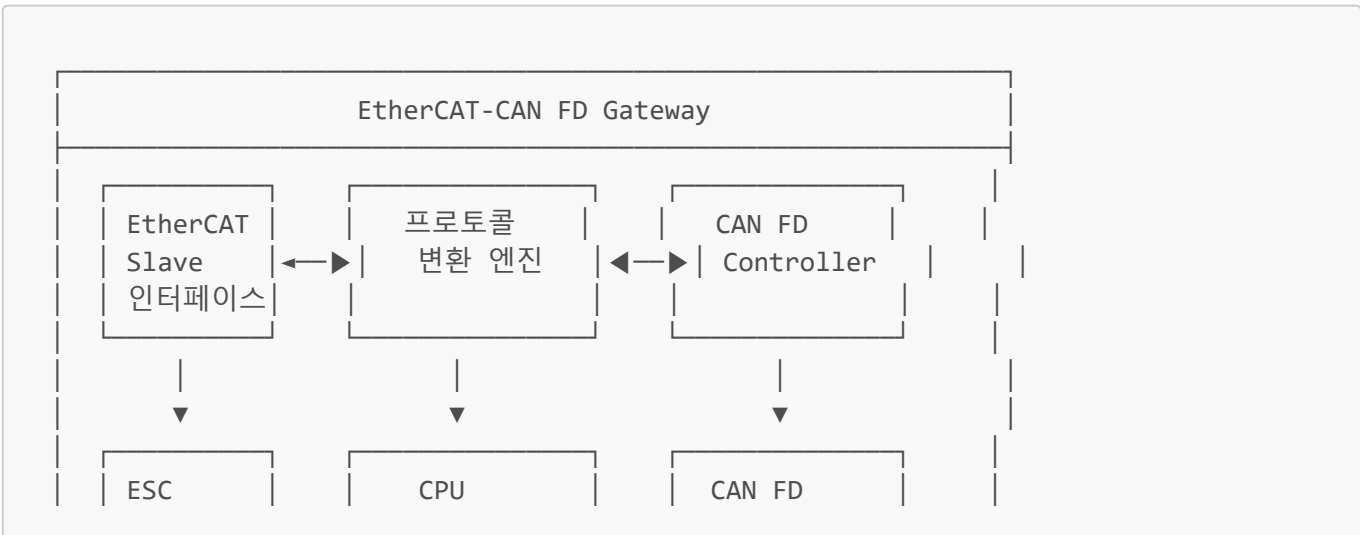


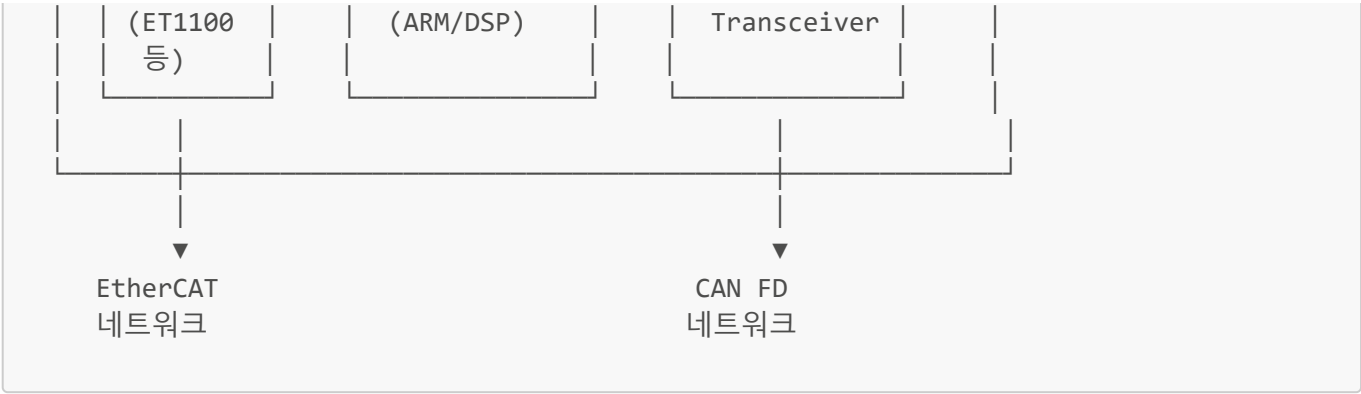
연결이 필요한 시나리오:

- 1. 기존 CAN FD 장비를 EtherCAT 시스템에 통합
- 2. 자동차 생산라인에서 차량 ECU와 공장 자동화 연결
- 3. 분산 센서 네트워크를 중앙 제어 시스템에 통합
- 4. 레거시 CAN 장비의 점진적 마이그레이션

5.2 연결 방법 1: EtherCAT-CAN FD 게이트웨이

5.2.1 게이트웨이 구조





5.2.2 상용 게이트웨이 제품

제조사	제품명	특징
HMS Networks	Anybus X-gateway	EtherCAT ↔ CAN FD 양방향
Hilscher	netTAP	다중 프로토콜 지원
PEAK-System	PCAN-Gateway	CAN FD 전문
Ixxat	CAN@net NT	유연한 구성
Beckhoff	EL6751	EtherCAT 터미널 형태

5.2.3 게이트웨이 설정 예시

```
<!-- EtherCAT-CAN FD Gateway 설정 예시 -->
<Gateway>
  <EtherCAT>
    <SlaveAddress>1</SlaveAddress>
    <CycleTime>1ms</CycleTime>
  </EtherCAT>

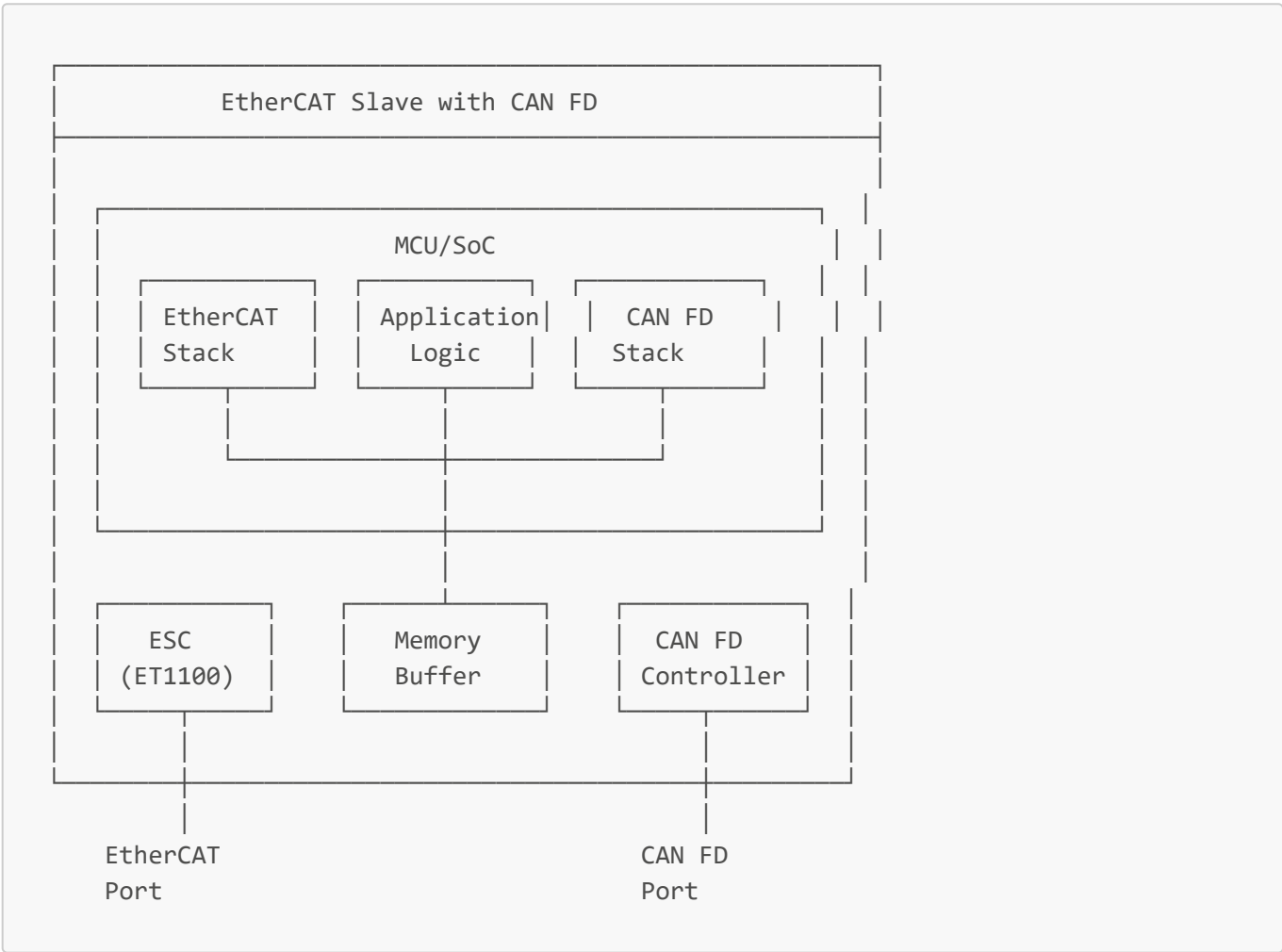
  <CANFD>
    <Bitrate>
      <Arbitration>500000</Arbitration> <!-- 500 kbps -->
      <Data>2000000</Data> <!-- 2 Mbps -->
    </Bitrate>
    <SamplePoint>80</SamplePoint>
  </CANFD>

  <Mapping>
    <!-- CAN FD 메시지 → EtherCAT PDO 매핑 -->
    <CANtoEtherCAT>
      <Message ID="0x100" DLC="16">
        <Map Offset="0" Size="16" PDO="TxPDO1"/>
      </Message>
      <Message ID="0x200" DLC="32">
        <Map Offset="0" Size="32" PDO="TxPDO2"/>
      </Message>
    </CANtoEtherCAT>
  </Mapping>
</Gateway>
```

```
<!-- EtherCAT PDO -> CAN FD 메시지 매핑 -->
<EtherCATtoCAN>
  <PDO Name="RxPDO1" Size="16">
    <Message ID="0x180" DLC="16"/>
  </PDO>
</EtherCATtoCAN>
</Mapping>
</Gateway>
```

5.3 연결 방법 2: EtherCAT 슬레이브 with CAN FD 인터페이스

5.3.1 통합 슬레이브 구조

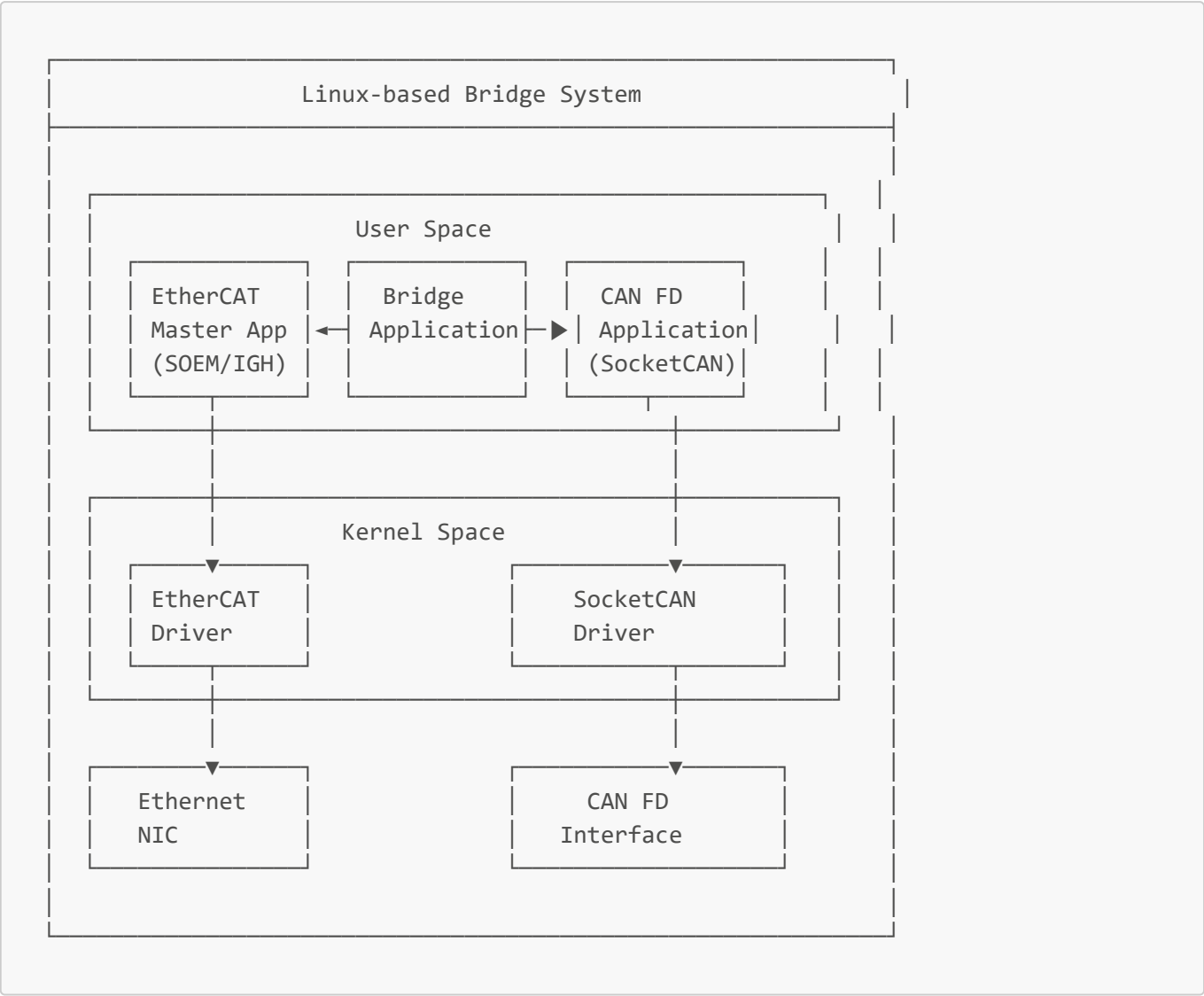


5.3.2 주요 칩셋 조합

ESC	MCU	CAN FD Controller	비고
ET1100	STM32H7	내장 FDCAN	고성능
LAN9252	ESP32-S3	TWAI (CAN 2.0)	저비용
AX58100	TMS570	내장 MCAN	산업용
XMC4800	내장	내장 MultiCAN	올인원

5.4 연결 방법 3: 소프트웨어 기반 브리지

5.4.1 Linux 기반 브리지 구현



5.4.2 샘플 코드 (Python)

```
#!/usr/bin/env python3
"""
EtherCAT - CAN FD Bridge Example
Using SOEM (Simple Open EtherCAT Master) and python-can
"""

import can
import ctypes
import threading
import time
from dataclasses import dataclass

# CAN FD 설정
CANFD_CHANNEL = 'can0'
CANFD_BITRATE = 500000
```

```

CANFD_DBITRATE = 2000000

# EtherCAT 설정 (SOEM 라이브러리 사용)
ETHERCAT_INTERFACE = 'eth0'

@dataclass
class BridgeMessage:
    """브리지 메시지 구조"""
    source: str # 'ethercat' or 'canfd'
    data: bytes
    timestamp: float

class EtherCATCANFDBridge:
    def __init__(self):
        self.running = False
        self.canfd_bus = None
        self.message_queue = []

    def init_canfd(self):
        """CAN FD 인터페이스 초기화"""
        self.canfd_bus = can.interface.Bus(
            channel=CANFD_CHANNEL,
            bustype='socketcan',
            fd=True,
            bitrate=CANFD_BITRATE,
            data_bitrate=CANFD_DBITRATE
        )
        print(f"CAN FD initialized: {CANFD_CHANNEL}")

    def init_ethercat(self):
        """EtherCAT 마스터 초기화 (SOEM 사용)"""
        # SOEM 라이브러리 로드
        self.soem = ctypes.CDLL('libsoem.so')

        # 인터페이스 초기화
        ifname = ETHERCAT_INTERFACE.encode()
        if self.soem.ec_init(ifname) <= 0:
            raise Exception("EtherCAT init failed")

        # 슬레이브 스캔
        if self.soem.ec_config_init(0) <= 0:
            raise Exception("No EtherCAT slaves found")

        print(f"EtherCAT initialized: {ETHERCAT_INTERFACE}")

    def canfd_to_ethercat(self, can_msg):
        """CAN FD 메시지를 EtherCAT PDO로 변환"""
        # 예: CAN ID 0x100 → EtherCAT Slave 1, PDO offset 0
        if can_msg.arbitration_id == 0x100:
            slave_addr = 1
            pdo_offset = 0
            # SOEM을 통해 PDO 쓰기
            # self.soem.ec_slave[slave_addr].outputs[pdo_offset:] = can_msg.data

```

```

def ethercat_to_canfd(self, slave_addr, pdo_data):
    """EtherCAT PDO 데이터를 CAN FD 메시지로 변환"""
    # 예: Slave 1, PDO → CAN ID 0x180
    if slave_addr == 1:
        can_msg = can.Message(
            arbitration_id=0x180,
            data=pdo_data,
            is_fd=True,
            bitrate_switch=True
        )
        self.canfd_bus.send(can_msg)

def canfd_receive_thread(self):
    """CAN FD 수신 스레드"""
    while self.running:
        msg = self.canfd_bus.recv(timeout=0.1)
        if msg:
            self.canfd_to_ethercat(msg)

def ethercat_cycle_thread(self):
    """EtherCAT 사이클 스레드"""
    while self.running:
        # EtherCAT 사이클 실행
        self.soem.ec_send_processdata()
        self.soem.ec_receive_processdata(1000) # 1ms timeout

        # PDO 데이터 읽어서 CAN FD로 전송
        # for slave in range(1, slave_count + 1):
        #     pdo_data = self.soem.ec_slave[slave].inputs[:]
        #     self.ethercat_to_canfd(slave, pdo_data)

        time.sleep(0.001) # 1ms 사이클

def start(self):
    """브리지 시작"""
    self.init_canfd()
    self.init_ethercat()

    self.running = True

    # 스레드 시작
    threading.Thread(target=self.canfd_receive_thread, daemon=True).start()
    threading.Thread(target=self.ethercat_cycle_thread, daemon=True).start()

    print("Bridge started")

def stop(self):
    """브리지 종료"""
    self.running = False
    if self.canfd_bus:
        self.canfd_bus.shutdown()
    self.soem.ec_close()
    print("Bridge stopped")

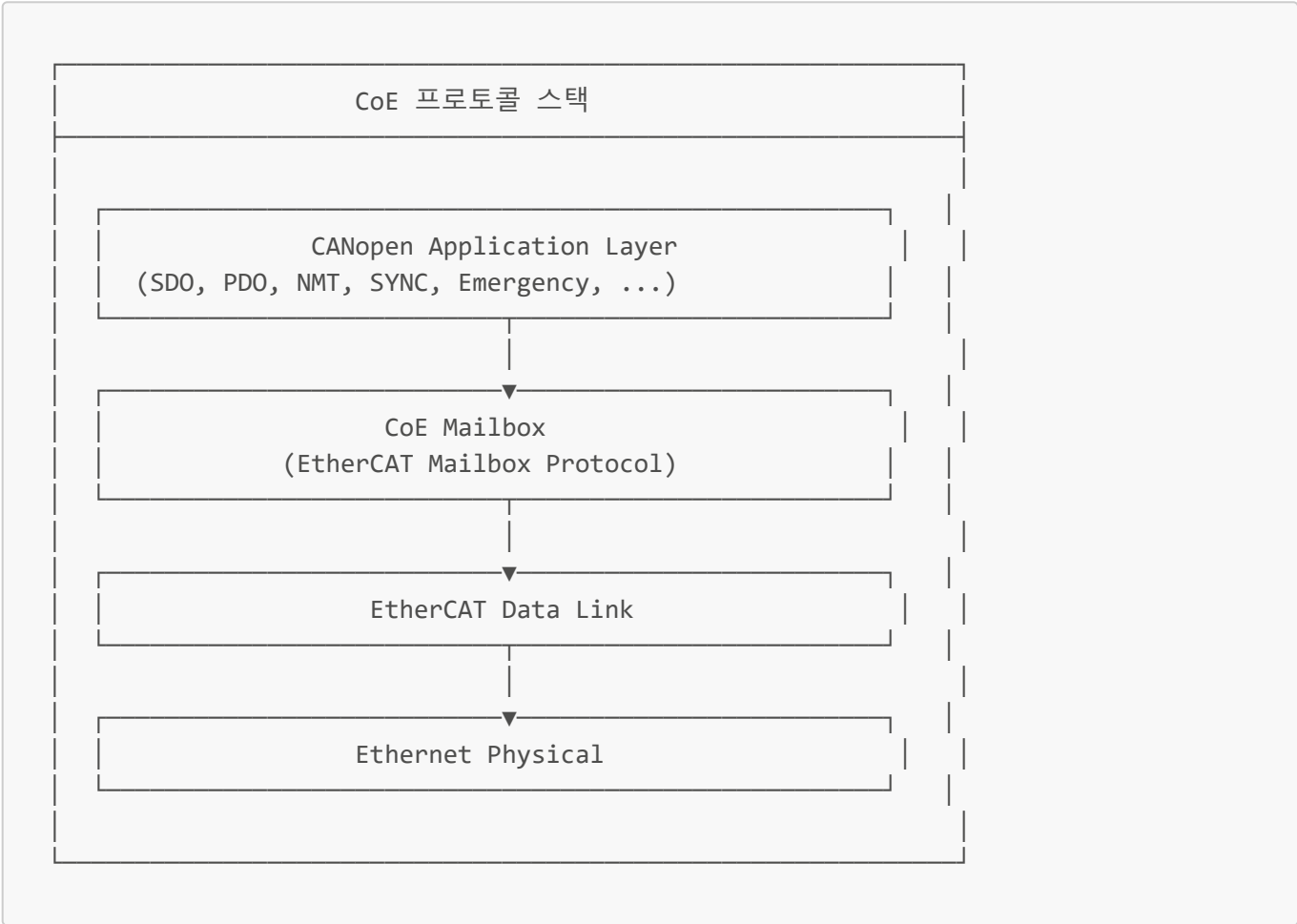
```

```
if __name__ == "__main__":
    bridge = EtherCATCANFDBridge()
    try:
        bridge.start()
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        bridge.stop()
```

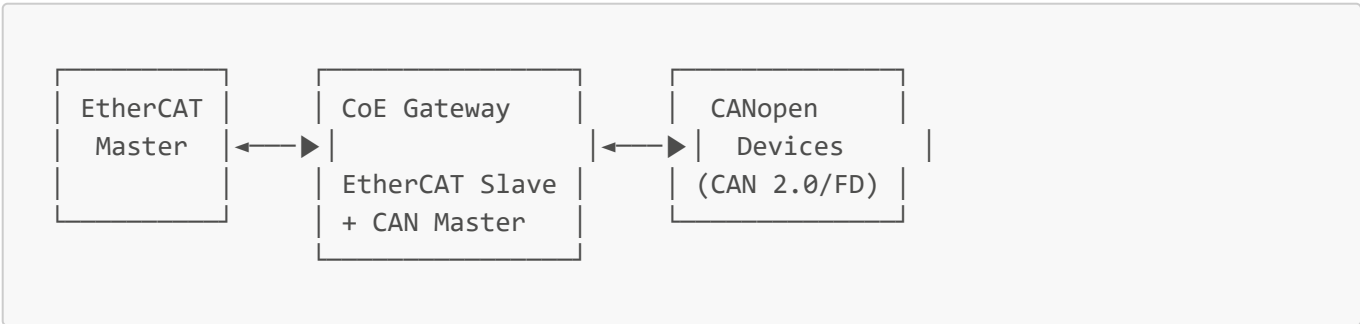
5.5 연결 방법 4: CoE (CANopen over EtherCAT)

5.5.1 CoE 개념

CoE는 CANopen 프로토콜을 EtherCAT 위에서 실행하는 방식입니다.

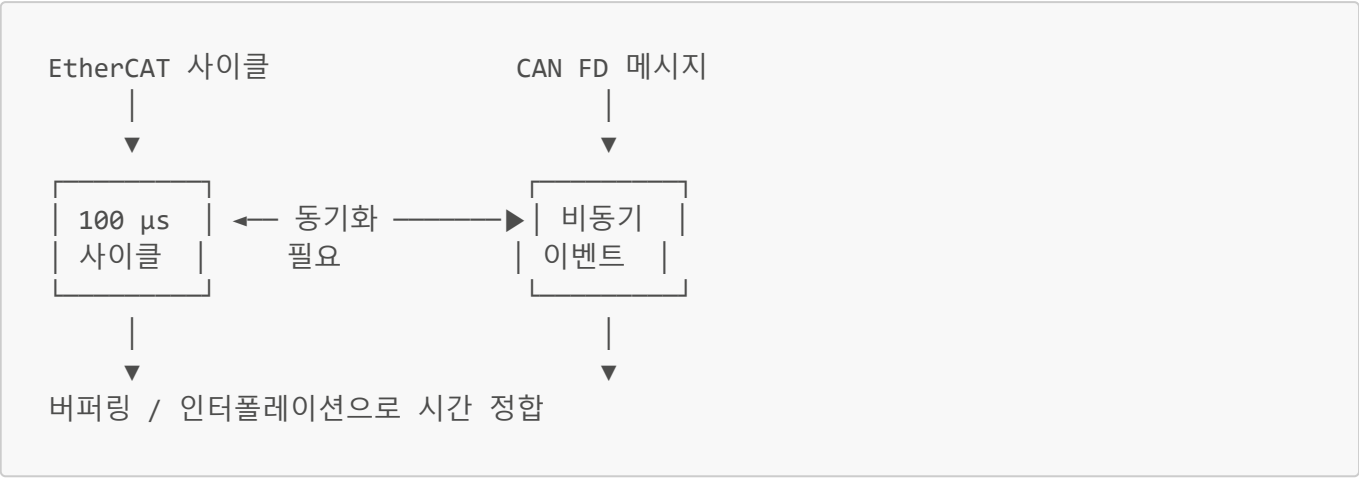


5.5.2 CoE를 통한 CAN 장비 통합



5.6 연결 시 고려사항

5.6.1 타이밍 동기화



5.6.2 데이터 매핑 전략

EtherCAT	CAN FD	매핑 방식
PDO (Process Data)	주기적 메시지	직접 매핑
SDO (Service Data)	비주기 메시지	요청/응답
Mailbox	진단 메시지	이벤트 기반

5.6.3 에러 처리

에러 처리 전략

1. CAN FD 에러 → EtherCAT Emergency 메시지로 전파

2. EtherCAT 통신 두절 → CAN FD 노드에 안전 상태 명령

3. 게이트웨이 자체 감시 (Watchdog)

- 양쪽 네트워크 상태 모니터링

- 타임아웃 시 안전 동작

4. 재연결 자동화

- CAN FD 버스오프 복구

- EtherCAT 슬레이브 재초기화

A. 용어 정리

용어	설명
ESC	EtherCAT Slave Controller
PDO	Process Data Object (실시간 데이터)
SDO	Service Data Object (설정 데이터)
CoE	CANopen over EtherCAT
FoE	File access over EtherCAT
FSoE	Failsafe over EtherCAT
BRS	Bit Rate Switch (CAN FD)
DLC	Data Length Code
ECU	Electronic Control Unit

B. 참고 자료

- EtherCAT Technology Group: <https://www.ethercat.org>
- CAN in Automation (CiA): <https://www.can-cia.org>
- Beckhoff Information System: <https://infosys.beckhoff.com>
- SOEM (Simple Open EtherCAT Master): <https://github.com/OpenEtherCATsociety/SOEM>
- python-can: <https://python-can.readthedocs.io>

C. 관련 표준

표준	내용
IEC 61158	산업용 통신 네트워크 (EtherCAT 포함)
IEC 61784	산업용 통신 프로파일
ISO 11898-1	CAN 데이터 링크 계층 (CAN FD 포함)
ISO 11898-2	CAN 고속 물리 계층
ISO 15765	CAN 진단 통신

작성일: 2026-02-14 작성자: Claude